

Clearance & overrule cases

Every combination of clearance status × override that can actually occur, tied to the real tiering in `_clearance_move_priority` and the release/acquire rules.

The four priority tiers (for winning new stock)

Tier	Condition	Can acquire new stock?
0	<code>clearance_defer_reason == "Scheduled Future Stock"</code>	Yes — outranks everything, including a fresh force-reserve
1	<code>is_force_reserved</code>	Yes — ahead of all ordinary clearance-tier orders
2	<code>order.clearance_date</code> set	Yes — by timestamp, earliest first
3	none of the above	Never

Hard lock is **not** a tier at all — it never appears in this ranking.

Every clearance × overrule combination that matters

1. **Cleared, no override** — ordinary tier 2. Can win stock and can lose it again to an earlier-clearance competitor on any later run.
2. **Uncleared (no_invoice), no override** — tier 3. Never wins anything through the queue. If it's somehow holding stock (leftover data, or native's own Reserve link bypassing this module entirely), it's a "foreign" holder — reclaimed unconditionally on the next run.
3. **Force-Reserved, cleared** — tier 1. Outranks ordinary paid competitors for new stock.
4. **Force-Reserved, uncleared (no_invoice) — still tier 1.** `is_force_reserved` is checked independently of `fulfillment_stage` — a `no_invoice` order that's *only* force-reserved still competes ahead of every genuinely paid order. This is deliberate (force-reserve is an active override, unlike hard lock) but easy to forget.
5. **Hard-Locked (order or line), currently holding stock** — protected unconditionally from release, regardless of tier, regardless of what anyone else's clearance timestamp says. The one override that can never be reclaimed by anything.
6. **Hard-Locked only, no_invoice, holding nothing** — completely excluded from the active reservation pass. Can never acquire so much as one unit through the queue — hard lock grants zero acquisition ability. The only way it ever holds anything is via an external path (native's own manual Reserve button, or the `_action_assign()` auto-rebalance hook).
7. **Hard-Locked, cleared, not yet holding** — competes normally at tier 2 to win stock; once won, is now also protected from ever losing it, even to a later order with a genuinely earlier clearance timestamp — the one case where clearance priority stops mattering for that specific stock.
8. **Hard-Locked and Force-Reserved together** — tier 1 for acquiring, plus unconditional protection for keeping. The maximally protected combination.
9. **Scheduled Future Stock holder (tier 0)** — note this applies to every allocation decision for that line while flagged, not just the one promised shipment. If unrelated stock for the same product frees up before the promised PO lands, tier 0 wins that too.
10. **Far-future exclusion (>6 months out) with an override** — the far-future exclusion explicitly does not apply if the line/order is hard-locked or force-reserved. An override always sticks regardless of how far out the order is scheduled.
11. **Order-wide hard lock vs. line-level force-reserve, different products** — order-level hard lock protects every product on that order; force-reserve is always line/product-scoped. A force-reserve on product A

never gives priority for product B on the same order, and vice versa for the queue-ranking (never order-wide bucket contamination in the actual `_clearance_move_priority` , only in the UI's own summary bucket).

12. **Cleared → demoted to no_invoice (full refund), while still hard-locked**— hard lock keeps protecting whatever it already holds, even though the order has genuinely lost its clearance entitlement. This is deliberate but worth knowing: hard lock's protection doesn't re-check `fulfillment_stage` at release time.
13. **Grace-period order, force-reserved, grace period expires unpaid (auto-demoted to no_invoice)**— the order loses its tier-2 claim (`clearance_date` cleared), but the force-reserved line keeps its tier-1 claim regardless, for the same reason as #4.
14. **Manual clearance override (`clearance_is_override=True`)** — behaves identically to a genuine payment for every tier/override rule above; the flag only affects display, never queue behavior.
15. **Order loses clearance (full refund, or a manual override back to no_invoice), then genuinely re-clears later** — `clearance_date_backup` saves the original timestamp before wiping `clearance_date` , and `_resolve_clearance_date` restores that exact backed-up value (not a fresh "now" stamp) the moment the order becomes eligible again — whether that's a real payment resuming or another manual override forward. The order keeps its original place in the queue rather than going to the back of the line as if it were brand new.

The one path where this deliberately does not happen: grace-period expiry. `_cron_expire_grace_period` explicitly clears `clearance_date_backup = False` rather than saving it — an order that simply never paid within the 15-day window loses its spot for good; if it's paid later, it starts completely fresh with a new timestamp, not its original confirmation-time one. The distinction is real payment/clearance being taken away (worth preserving) versus never actually earning it in the first place (nothing to restore).

Note: this is how it's implemented today, but it's worth revisiting — whether a manual admin override back to `no_invoice` should preserve and restore the original timestamp (current behavior) or start fresh instead, the way grace-period expiry does, is a real product question, not yet fully settled. Changing it later is low-risk either way: the manual-override `write()` block in `sale_order.py` would just switch `"clearance_date_backup": order.clearance_date` to `"clearance_date_backup": False` , mirroring `_cron_expire_grace_period` 's own pattern. The restore side (`_resolve_clearance_date`) is already generic — it doesn't know or care which path populated the backup, it just uses it if present. No ripple effects elsewhere.